



1. Problem Title & Link

Title: LeetCode 441 – Arranging Coins

Link: <https://leetcode.com/problems/arranging-coins/>

2. Problem Statement (Short Summary)

You have n coins.

Arrange them into a staircase such that:

- The first row has **1** coin.
- The second row has **2** coins.
- The third row has **3** coins.
- ...

Return the **number of complete rows** that can be formed.

3. Examples (Input → Output)

Example 1

Input

$n = 5$

Output

2

Explanation

```
*  
* *
```

Remaining coins = 2

The third row needs 3 coins but only 2 remain.

Example 2

Input

$n = 8$

Output

3

Explanation

```
*  
* *  
* * *
```

Remaining = 2



Example 3

Input

$n = 1$

Output

1

4. Constraints

- $1 \leq n \leq 2^{31} - 1$
- Return the maximum number of complete rows.

5. Core Concept (Pattern / Topic)

- Binary Search
- Mathematical Formula
- Monotonic Search Space

6. Thought Process (Step-by-Step Explanation)

Brute Force

Build rows one by one.

Need 1 coin

Need 2 coins

Need 3 coins

...

Stop when coins are insufficient.

Time Complexity

$O(\sqrt{n})$

Since

$$1+2+3+\dots+k \approx \frac{k^2}{2}$$

Optimized Thinking

Suppose we build k complete rows.

Coins needed are

$$1 + 2 + 3 + \dots + k$$

Using the arithmetic progression formula,

$$\frac{k(k+1)}{2}$$



We need the **largest k** satisfying

$$k(k+1)/2 \leq n$$

Notice:

As k increases,

Coins Needed

1

3

6

10

15

21

...

This increases monotonically.

Hence Binary Search.

Intuition

We are searching for

Largest valid k

where

$$k(k+1)/2 \leq n$$

Binary Search efficiently finds this value.

Why This Works

The function

$$f(k) = k(k+1)/2$$

is strictly increasing.

Once it exceeds n, all larger values are also invalid.

This monotonic property makes Binary Search applicable.

7. Visual / Intuition Diagram (ASCII)

Example

n = 8

Rows

1 → Total = 1

2 → Total = 3



3 → Total = 6

4 → Total = 10 **✗**

Binary Search

1	2	3	4
---	---	---	---

✓	✓	✓	✗
---	---	---	---

Answer = 3

8. Pseudocode (Language Independent)

```

left = 1
right = n

while left <= right

    mid

    coins = mid*(mid+1)/2

    if coins <= n
        answer = mid
        search right

    else
        search left

return answer

```

9. Code Implementation

Python

```

class Solution:
    def arrangeCoins(self, n: int) -> int:

        left, right = 1, n
        ans = 0

        while left <= right:

            mid = (left + right) // 2
            coins = mid * (mid + 1) // 2

            if coins <= n:
                ans = mid
                left = mid + 1
            else:

```



```
        right = mid - 1

    return ans
```

Java

```
class Solution {

    public int arrangeCoins(int n) {

        long left = 1;
        long right = n;
        long ans = 0;

        while (left <= right) {

            long mid = left + (right - left) / 2;

            long coins = mid * (mid + 1) / 2;

            if (coins <= n) {
                ans = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return (int) ans;
    }
}
```

Note: We use long in Java to avoid integer overflow while computing $\text{mid} * (\text{mid} + 1)$.

10. Time & Space Complexity

Time Complexity

$O(\log n)$

Binary Search halves the search space each iteration.

Space Complexity

$O(1)$

Only constant extra variables are used.

11. Common Mistakes / Edge Cases

- Using int in Java can overflow for large n.
- Forgetting to store the last valid answer.



- Using < instead of <=.
- Binary searching for the first invalid value instead of the last valid value.

12. Detailed Dry Run

Example

n = 8

Left	Right	Mid	Coins Needed	Valid ?	Answer
1	8	4	10	✗	0
1	3	2	3	✓	2
3	3	3	6	✓	3

Loop ends.

Output

3

13. Common Use Cases (Real-Life / Interview)

- Resource allocation
- Capacity planning
- Largest feasible value search
- Scheduling problems
- Binary Search on Answer

14. Common Traps (Important!)

- Integer overflow.
- Wrong Binary Search condition.
- Forgetting to move left = mid + 1 after a valid answer.
- Returning left instead of the last valid mid.

15. Builds To (Related LeetCode Problems)

- LC 69 – Sqrt(x)
- LC 367 – Valid Perfect Square
- LC 875 – Koko Eating Bananas
- LC 1011 – Capacity To Ship Packages Within D Days
- LC 1283 – Find the Smallest Divisor Given a Threshold

16. Alternate Approaches + Comparison



Approach	Time	Space	Notes
Simulation	$O(\sqrt{n})$	$O(1)$	Easy but slower
Binary Search	$O(\log n)$	$O(1)$	Standard interview solution
Mathematical Formula	$O(1)$	$O(1)$	Fastest, requires derivation

Mathematical Formula

From

$$k(k+1)/2 \leq n$$

Solving the quadratic equation gives

$$k = \lfloor (\sqrt{8n + 1} - 1) / 2 \rfloor$$

This computes the answer directly in constant time.

17. Why This Solution Works (Short Intuition)

The number of coins required for k complete rows is $k(k+1)/2$, which increases monotonically with k . Binary Search efficiently finds the largest k whose required coins do not exceed n .

18. Variations / Follow-Up Questions

1. Solve using the mathematical formula instead of Binary Search.
2. Generalize for row sizes increasing by a constant other than 1.
3. Given k rows, determine the remaining unused coins.
4. Extend the problem where each row size follows a custom sequence.
5. Find the minimum coins needed to complete the next row.
- 6.